PrepBench Instructional Standard

VERSION 1.3 | REUSABLE PRODUCT STANDARD

Status	Approved baseline — approved 2026-10-13
Scope	All PrepBench learning packs — ADF, Databricks, Agentic AI, and future packs — including roadmaps, study guides, learning units, tutorials, labs, and assessments.

Change log

Version 1.3 closes three gaps identified in the v1.2 review: independent verification of beginner comprehension, clearer conformance classification (including reference-only material), and a mandatory pack-annex template. It also establishes a pack-level overview visual requirement inspired by the role of Microsoft's ADF Visual Guide, clarifies cross-pack terminology, and strengthens the publishing quality gate.

1. Purpose

PrepBench must help a learner progress from unfamiliarity to understanding, from understanding to practical competence, and from practical competence to independent technical judgment.

The platform must not assume that a learner already understands a concept simply because the concept is included in a technical curriculum.

Core principle: Explain why a concept exists, what it means, how it works, and how to use it before expecting the learner to evaluate its advanced behaviors or trade-offs.

Technical correctness is mandatory. Beginner comprehension is equally mandatory.

This standard is pack-agnostic. It governs how every PrepBench learning pack translates its roadmap into a learning experience. Pack-specific assumptions, source hierarchies, terminology, visual assets, and lab requirements are defined in a mandatory pack annex; the instructional rules in this document do not vary by pack.

2. Target learner and assumptions

The default learner is a technically curious beginner or developing practitioner who may understand general computing, databases, or data concepts but may have little or no hands-on experience with the specific technology being taught.

Every curriculum must explicitly define its assumed prerequisites.

- Never assume familiarity with a product-specific term, feature, architecture, acronym, or configuration just because it appears in the roadmap.
- Introduce unfamiliar terminology in context before relying on it.
- Expand acronyms on first use and explain why the term matters.
- Use prerequisites to identify knowledge genuinely needed before a lesson, not to excuse unexplained terminology.
- Provide an accessible entry point while retaining optional advanced material for experienced learners.

The learner should not need to open several unrelated documentation pages simply to understand the first paragraph of a lesson.

3. The required learning sequence

Every major concept must follow this progression. The amount of detail may vary by complexity, but the instructional logic must be preserved. Stages may be combined in short units or split into separate units for complex concepts.

Stage 1 — Establish the problem

Begin with the real-world situation, business need, technical challenge, or failure that makes the concept necessary. Answer: What problem are we trying to solve, and why should the learner care?

Stage 2 — Introduce the concept

Explain the concept in plain English and provide an intuitive mental model. Define unfamiliar terms before using them to explain other terms. Analogies may help, but must not replace accurate technical explanations.

Stage 3 — Explain how it works

Describe the main components, their relationships, execution sequence, and relevant boundaries. Use the simplest visual or example that materially improves comprehension. Explain the visual rather than assuming the learner can interpret it independently.

Use a mind map or equivalent hierarchical visual only when multiple interdependent parts, decision branches, or relationships cannot be made clear with a simpler diagram or linear explanation. Do not add mind maps by default.

Stage 4 — Demonstrate a realistic example

Walk through a representative scenario. Explain important configuration, decisions, expected behavior, and outcomes. Where code or configuration is involved, show enough context for the learner to understand where it belongs and why it is needed.

Stage 5 — Guided practice and feedback

Give the learner an opportunity to apply the idea. Start with guidance appropriate to the learner's experience, then reduce that guidance as the learner gains competence. Explain why an answer or implementation is correct or incorrect, rather than merely reporting a score.

Stage 6 — Extend to real engineering practice

After the core concept is understood, introduce relevant limitations, troubleshooting, security, performance, cost, operational concerns, and architectural trade-offs. Do not remove advanced material to make a lesson easier; introduce it when the learner has the foundation to understand it.

Stage 7 — Consolidate learning

End with a concise summary, key ideas to remember, a common mistake or misconception, an appropriate knowledge check, and a clear next step.

4. Required lesson design

Each substantive learning unit must define the following elements in its content or its instructional metadata.

Element	Requirement
Learning objective	State what the learner will be able to explain, create, diagnose, compare, or decide.
Prerequisites	State essential prior knowledge, or explicitly say that none is required.
Context	Explain the problem, use case, or motivation.
Concept explanation	Introduce the idea clearly, defining unfamiliar terms in context.
Worked example	Show how the concept behaves in a realistic scenario.
Application	Include an exercise, practical task, or worked application appropriate to the objective. A full lab is not required where it adds no value.
Understanding check	Test the intended learning outcome and provide explanatory feedback.
Summary	Consolidate the principal takeaways and direct the learner to the next step.
Sources	Link to authoritative documentation for claims that require verification.

A unit that does not teach an applicable skill does not need to pretend to have a hands-on objective. It still needs a meaningful way to check understanding.

5. Progressive disclosure: core first, depth second

Content must be organized into three instructional levels.

Level A — Understand: Build a mental model, teach the vocabulary, explain why the feature exists, and show how it works at a basic level.

Level B — Apply: Walk through an example, configure or use the feature where relevant, interpret the result, and practice the common use case.

Level C — Engineer: Explore edge cases, documented limits, failure behavior, security requirements, operational constraints, and advanced trade-offs.

These levels need not appear as literal headings in every lesson, but the content sequence must reflect them. Where content is long or complex, divide it into meaningful units with distinct outcomes and checkpoints. Do not use arbitrary word counts as the primary measure of lesson size.

6. Pack-level overview visual: the complete mental model

Every learning pack must include a **Pack Overview Visual** near the beginning of its introductory learning path. Its purpose is to give the learner a coherent, at-a-glance mental model of the technology or domain before they study components in depth.

Microsoft's Azure Data Factory Visual Guide (https://learn.microsoft.com/en-us/azure/data-factory/media/introduction/data-factory-visual-guide-small.png), presented from the ADF introduction (https://learn.microsoft.com/en-us/azure/data-factory/introduction), is a reference for the *instructional function* of such a visual: it helps learners see how the technology, its components, and the end-to-end process fit together. PrepBench must create an original, source-verified visual synthesis; do not copy Microsoft's artwork or layout verbatim.

The Pack Overview Visual must cover, at the appropriate level for the technology:

- **Why it exists:** the problem or need the technology addresses.
- **Where it fits:** the technology's role and important boundaries in a larger solution.
- **Anchor scenario:** one representative use case that gives the visual a simple story and shows how the main parts cooperate.
- **Core building blocks:** the principal components and how they relate.
- **End-to-end flow:** how inputs, processing/orchestration, outputs, and consumers connect, or the equivalent lifecycle for non-data technologies.
- **Execution and integration:** where work happens and which external systems, services, or agents participate, where relevant.
- **Cross-cutting concerns:** security, governance, observability, deployment, or evaluation where these are central to the domain.
- **Outcome and value:** what the complete system or workflow enables and the key value it provides.

For ADF, the overview should help a beginner connect data sources, pipelines and activities, linked services and datasets, integration runtimes, transformation/compute services, destinations, triggers, monitoring, and deployment/governance at a high level. Trace one simple end-to-end scenario (for example, a file arrives, a trigger starts a pipeline, data is ingested, transformation or external compute runs, output is published, and execution is monitored). It should communicate relationships and the end-to-end story rather than attempt to list every property, connector, or limit.

Visual design and learner use

- Place the visual early, after a short orientation that states what the learner is looking at and why it matters.
- Use it as a **pre-read and post-review anchor**: introduce it before detailed lessons, revisit it as the learner encounters components, and use it to consolidate the big picture.
- Prioritize a legible overview over a crowded poster. If full detail cannot remain readable at normal display size, provide a high-resolution or zoomable detailed companion, while keeping the main overview coherent.
- Label components and directional relationships clearly. Use arrows with meaning, a clear visual hierarchy, restrained color, and a legend when needed.
- Provide useful alternative text and a caption that explains the diagram's scope and how the learner should read it.
- Add source attribution and confirm reuse rights for every reused third-party asset. Do not assume that an image is reusable merely because it appears on a documentation site or in a repository; verify the specific asset's terms. Prefer original PrepBench diagrams built from verified facts.
- Keep the visual subordinate to accurate explanation: the visual summarizes and connects ideas; it does not replace the sequential lesson, glossary, worked examples, or detailed documentation.

Each substantive chapter should also include a concept-specific diagram or other visual where it materially improves understanding. A separate diagram is not required when a short explanation or example is clearer. Mind maps remain selective, not a default format.

7. Authoritative sources and technical accuracy

Every learning pack must define its source hierarchy in its pack annex: authoritative documentation for the technology it teaches (for example, Microsoft Learn for Microsoft products, Databricks documentation for Databricks, and primary research plus official model/provider documentation for Agentic AI topics).

- Verify authoritative documentation for the claim being taught.
- Preserve important technical qualifications and product-, connector-, or model-specific limitations.
- Distinguish documented vendor behavior from PrepBench examples, interpretations, recommendations, and architectural opinions.
- Attribute direct quotations and keep them brief. Use original explanations and examples to teach the concept.
- Record source links and verify time-sensitive behavior or limits when content is updated.
- Where documentation is ambiguous, inconsistent, or silent, state the uncertainty and identify what needs validation. Do not silently invent an answer.
- Never replace technical precision with an oversimplified statement that becomes misleading.

Source accuracy is necessary, but a source citation is not a substitute for teaching. A correct definition followed by a link does not automatically constitute a complete lesson.

8. Examples, demonstrations, and labs

Examples must teach a transferable concept rather than merely display a working configuration.

Every substantive practical exercise should specify, as applicable:

1. The task and its purpose.
2. Required inputs, prerequisites, permissions, and resources.
3. Steps or constraints for completing the task.
4. The expected output or observable behavior.
5. How to verify success.
6. Common failure modes and how to diagnose them.
7. Cleanup, security, and potential cost considerations.

A guided example should explain meaningful decisions. Copying a JSON block, command, or configuration without understanding its purpose must not be treated as demonstrated competence.

Learning Lab scenarios should map to relevant learning objectives and progress from guided practice toward independent application.

9. Assessment must match the learning objective

A lesson's assessment must test what the lesson claims to teach.

- **Explain:** require an explanation or an appropriate conceptual question.
- **Configure:** require the learner to configure or correct something.
- **Diagnose:** provide evidence from a scenario and require the learner to identify the cause.
- **Compare:** require a reasoned choice with explicit trade-offs.
- **Design:** require a design artifact or scenario-based solution.

Recall questions have a role, particularly for terminology and limits, but must not be the sole evidence of practical competence. Feedback must explain the reasoning behind the result and, where useful, direct the learner to the specific concept that needs review.

Evidence loop: Assessment outcomes are evidence about content quality, not only learner performance. Where the product captures results, recurring failure patterns on a unit must trigger content review. When learners consistently miss an item, the first question is whether the lesson taught what the assessment tests.

10. Learner experience and navigation

The learning interface must reinforce the teaching structure.

- Show the learning objective and expected outcome clearly.
- Keep the learner focused on the current concept.
- Use headings that describe the learning task or idea, not internal curriculum administration.
- Make visuals readable, contextual, accessible, and appropriately attributed.
- Use tables for real comparisons, decisions, limits, or structured reference information, not as a substitute for explanation.

- Make official references available without interrupting the main learning sequence.
- Provide clear progression to the next unit, relevant practice, and subsequent learning stage.
- Track lesson or unit progress where the product supports it.
- Keep curriculum traceability, topic IDs, coverage accounting, and audit metadata available to content maintainers without making them the dominant learner experience.
- Use consistent terminology when the underlying concept is the same. Where similar terms have different meanings across products, explain the distinction explicitly and cross-link the concepts.

The study guide should teach; the roadmap should organize; the Learning Lab should develop practical ability; the assessment should provide evidence of learning; the Pack Overview Visual should provide the integrated mental model.

11. Curriculum and prerequisite design

The roadmap remains the source of truth for curriculum coverage. Reorganizing material for better teaching must not silently remove required topics.

Each roadmap topic must map to one or more learning units and identify:

- Its intended learning outcomes.
- The prerequisite concepts the learner needs.
- Where the concept is taught in depth.
- Where it is practiced or assessed.
- Which authoritative sources support its technical claims.
- Where advanced details or related concepts are taught.

Dependencies should determine learning order. A prerequisite concept must be introduced before a lesson relies on it without explanation. Related advanced content can be cross-linked, but links must not replace the foundational explanation needed to understand the current unit.

12. Independent comprehension review and quality gate

An author cannot be the sole judge of whether their lesson is understandable to a beginner. Before a unit is approved, a reviewer who did not author it must inspect the unit against its declared learner profile and prerequisites.

For the **reference-chapter pilot** in each pack, at least two representative learners who are unfamiliar with the target product or domain should attempt to use the chapter with only its stated prerequisites and provided materials. Observe where they hesitate, misinterpret terms, cannot explain a relationship, or need outside material to understand the basic lesson. Record findings and resolve significant comprehension issues before accepting the reference chapter. This is a formative usability check, not a claim of statistically representative research.

For subsequent units, the independent reviewer must perform a beginner-comprehension pass. Practical or high-risk units should receive learner testing when the lesson introduces a new instructional pattern, repeated assessment failures indicate confusion, or the reviewer identifies unresolved comprehension risk. If a required pilot has not been conducted, mark the unit or chapter **provisional — comprehension not yet validated** rather than claiming it has passed the beginner-comprehension gate.

Publish checklist

1. Can the intended learner understand why the concept matters?
2. Are prerequisites explicit, and is unfamiliar terminology introduced before it is relied upon?
3. Is the explanation technically accurate and appropriately sourced?
4. Can the learner explain how the components fit together?
5. Is there a useful worked example or demonstration where appropriate?
6. Does the learner have an appropriate opportunity to apply or check the concept?
7. Is advanced content introduced after the necessary foundation?
8. Does feedback explain the reasoning behind success or failure?
9. Are navigation and the next learning step clear?
10. Does the unit meet all mapped curriculum requirements?
11. Has a reviewer other than the author completed the comprehension pass?
12. For pack introductions, is the Pack Overview Visual coherent, legible, accessible, source-verified, and placed early enough to orient the learner?
13. Have visual attribution and reuse terms been checked for the actual assets used?

Technical accuracy, source integrity, curriculum coverage, and the applicable comprehension review are mandatory gates. A strong overall checklist score cannot compensate for failure in any of these areas.

13. Conformance classification

Every instructional unit must be assigned one of these classifications in its pack annex or content metadata. Classification is based on the unit's intended learning outcome, not merely its length.

Core instructional unit

Use for a unit primarily intended to establish or reinforce understanding. It must define the objective and prerequisites, explain the concept in the required sequence, use an example when it materially helps, include an appropriate understanding check with explanatory feedback, cite relevant sources, and pass the quality gate. A full lab is not required when no hands-on outcome is claimed.

Full instructional unit

Required when the unit claims that a learner will independently apply, configure, build, operate, troubleshoot, diagnose, optimize, compare significant trade-offs, or design a solution; or when it includes a lab, practical exercise, or graded practical assessment. It must satisfy the relevant requirements in Sections 4, 8, 9, and 12, including observable success criteria and useful feedback.

Reference companion

Use for supplementary lookup material such as a connector matrix, limits reference, glossary, syntax catalogue, detailed configuration index, or advanced troubleshooting table that is not designed to teach a complete learning outcome by itself. A reference companion must still be technically accurate, sourced, readable, accessible, and cross-linked to the instructional unit that teaches the concept. It must not be presented as a substitute for that unit or claim a practical objective it does not assess.

A pack annex must identify each unit's classification. Any exceptional treatment must state its rationale; no classification or exception can waive requirements for technical accuracy, source integrity, accessibility, or the applicable quality gate.

14. Mandatory pack annex

Every pack must maintain an annex that specifies how this standard is applied to that technology. At minimum, it must contain:

1. **Learner profile and prerequisites:** intended audience, assumed baseline knowledge, and any required setup knowledge.
2. **Authoritative-source hierarchy:** primary sources by claim type, how conflicting sources are handled, and how changing/time-sensitive claims are re-verified.
3. **Terminology policy:** preferred terms, acronym conventions, glossary location, and known cross-product terminology differences.
4. **Lab environment and safeguards:** accounts, software/runtime versions, permissions, sample data, required infrastructure, estimated cost where material, cleanup, privacy, security, and safety expectations.
5. **Pack Overview Visual specification:** asset location, intended learning role, covered concepts, alternative text, caption, source/attribution and reuse-rights status, plus the high-resolution/zoom approach where needed.
6. **Curriculum mapping:** roadmap topics mapped to learning units, objectives, prerequisites, practice/assessment evidence, sources, and advanced/reference companion material.
7. **Conformance register:** each unit's Core, Full, or Reference companion classification, with a rationale for exceptions or reference-only treatment.
8. **Review plan:** independent-review owner or role, how comprehension findings are recorded, and triggers for re-review (such as source changes, recurring learner errors, or changes to labs).

An annex may add domain-specific fields but must not omit these minimum fields. Pack-specific requirements may add rigor; they cannot waive the cross-pack instructional rules.

15. How we will apply this standard to PrepBench

1. Ratify this standard as the cross-pack baseline and define or update the ADF pack annex using Section 14.
2. Create an original ADF Pack Overview Visual using Microsoft's guide as an instructional reference, not copying its artwork or layout; place it early in Chapter 1.
3. Rework ADF Chapter 1 into the required learner progression while preserving technical accuracy and complete roadmap coverage.

4. Conduct an independent comprehension review and test with at least two learners unfamiliar with ADF. Record and resolve significant issues before approving the reference chapter and annex.
5. Use the approved reference chapter to audit the remaining ADF chapters; apply the same standard to Databricks, Agentic AI, and future packs, creating an annex and overview visual for each.
6. Update content review checklists and relevant product tests so that the intended structure is maintained.